

2.4 动作 (Action) 通信

所属阶段： 第二阶段 · ROS 2 核心通信机制

讲次数量： 9 节

预计时长： ~60 分钟

前置依赖： 2.3 服务 (Service) 通信

学习目标

完成本节后，学员将能够：

- 解释 ROS 2 Action 的三段式结构 (Goal / Feedback / Result) 及其适用场景
- 理解 Action 底层由 3 个 Service + 2 个 Topic 组成，掌握 Goal 的完整生命周期
- 创建自定义 Action 定义文件 (`.action`)，完成接口包的构建与依赖配置
- 使用 Python (`rclpy.action`) 编写完整的 Action Server 和 Client
- 使用 C++ (`rclcpp_action`) 编写完整的 Action Server 和 Client
- 通过 Turtlesim 综合实验，实现"海龟移动到指定位置"的 Action 实战
- 运用 `ros2 action` 系列命令行工具查看、调试和手动发送 Goal
- 独立完成 C++ Action 编码挑战，综合运用 Action 通信知识

讲次 1：什么是 ROS 2 Action，为什么和何时使用

知识点

从 Service 到 Action：长时间任务的需求

在 2.3 中，我们学习了 Service——**同步的请求-响应**模式。Client 发送请求后阻塞等待，Server 处理完毕返回响应。这种模式适合“瞬间完成”的操作：

- 查询机器人电量 → 几毫秒内返回
- 触发相机拍照 → 单次操作，立即完成
- 重置仿真环境 → 一次性指令

但如果任务需要**数秒到数分钟**才能完成呢？

- 导航到目标点（途中需要知道“走了多远”）
- 机械臂抓取物体（需要实时报告“夹爪已闭合”）
- 电梯开门并等待乘客进入（需要持续反馈状态）

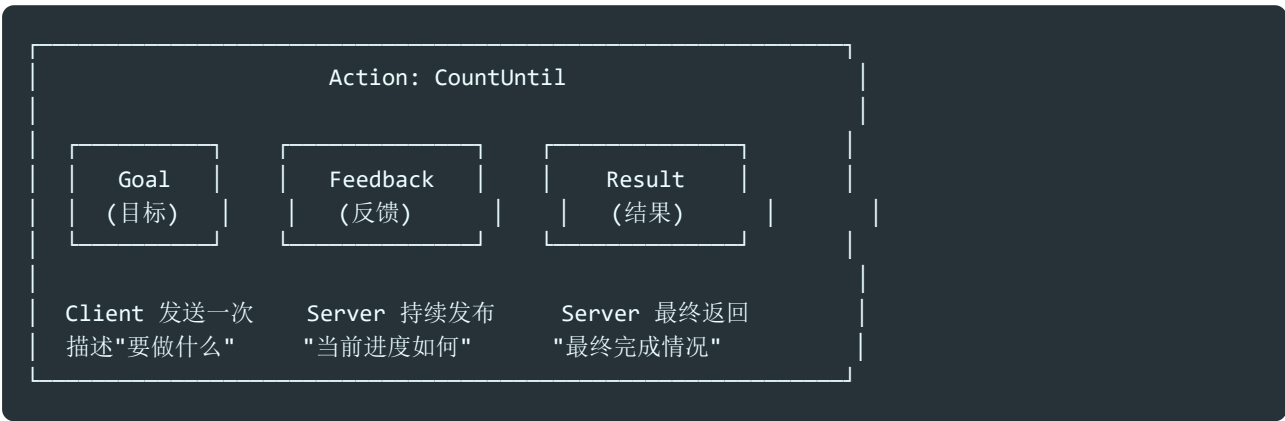
Service 的问题：

问题	说明
阻塞	Client 必须一直等待，无法做其他事
无进度	等待期间无法获知任务进展
难取消	中途想取消任务，Service 没有标准机制

Action（动作） 正是为解决这类“长时间任务 + 进度反馈 + 可取消”场景而设计的通信机制。

Action 的三段式结构

每一个 Action 由三个部分组成：



以本课程的 `CountUntil` Action 为例：

部分	字段	含义
Goal	target_number	要计数到的目标数字
Goal	period	每次计数间隔（秒）
Feedback	current_number	当前计数值（持续更新）
Result	reached_number	最终到达的数字

类比理解：

- **Goal** = 你告诉服务员"我要一份牛排，七分熟"（一次性指令）
- **Feedback** = 服务员每隔几分钟告诉你"正在煎"、"快好了"（持续更新）
- **Result** = 服务员端上成品"您的牛排好了"（最终结果）

Action 的底层实现

Action 并不是全新的传输协议，而是在 Topic 和 Service 之上构建的高层抽象。底层实际上由 **3 个 Service + 2 个 Topic** 组成：

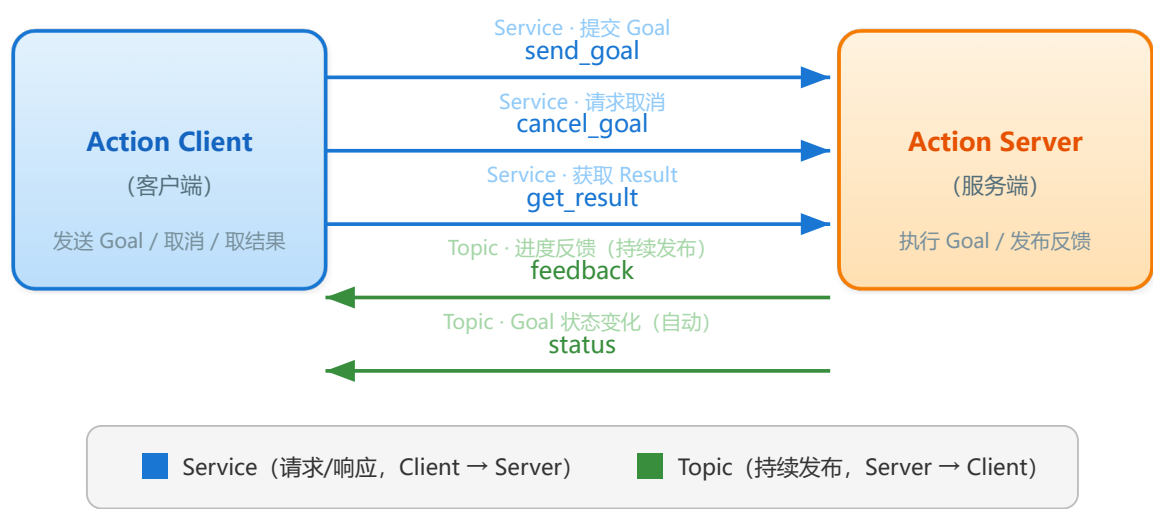
```

Action: /count_until
├── Service: /count_until/_action/send_goal      ← Client 发送 Goal 请求
├── Service: /count_until/_action/cancel_goal    ← Client 请求取消
├── Service: /count_until/_action/get_result     ← Client 获取最终 Result
├── Topic:   /count_until/_action/feedback      ← Server 周期性发布 Feedback
└── Topic:   /count_until/_action/status        ← Server 发布 Goal 状态变化（自动）

```

3 个 Service: send_goal（发送目标）、cancel_goal（请求取消）、get_result（获取结果）
2 个 Topic: feedback（进度反馈）、status（状态更新）

Action 底层结构：3 Service + 2 Topic

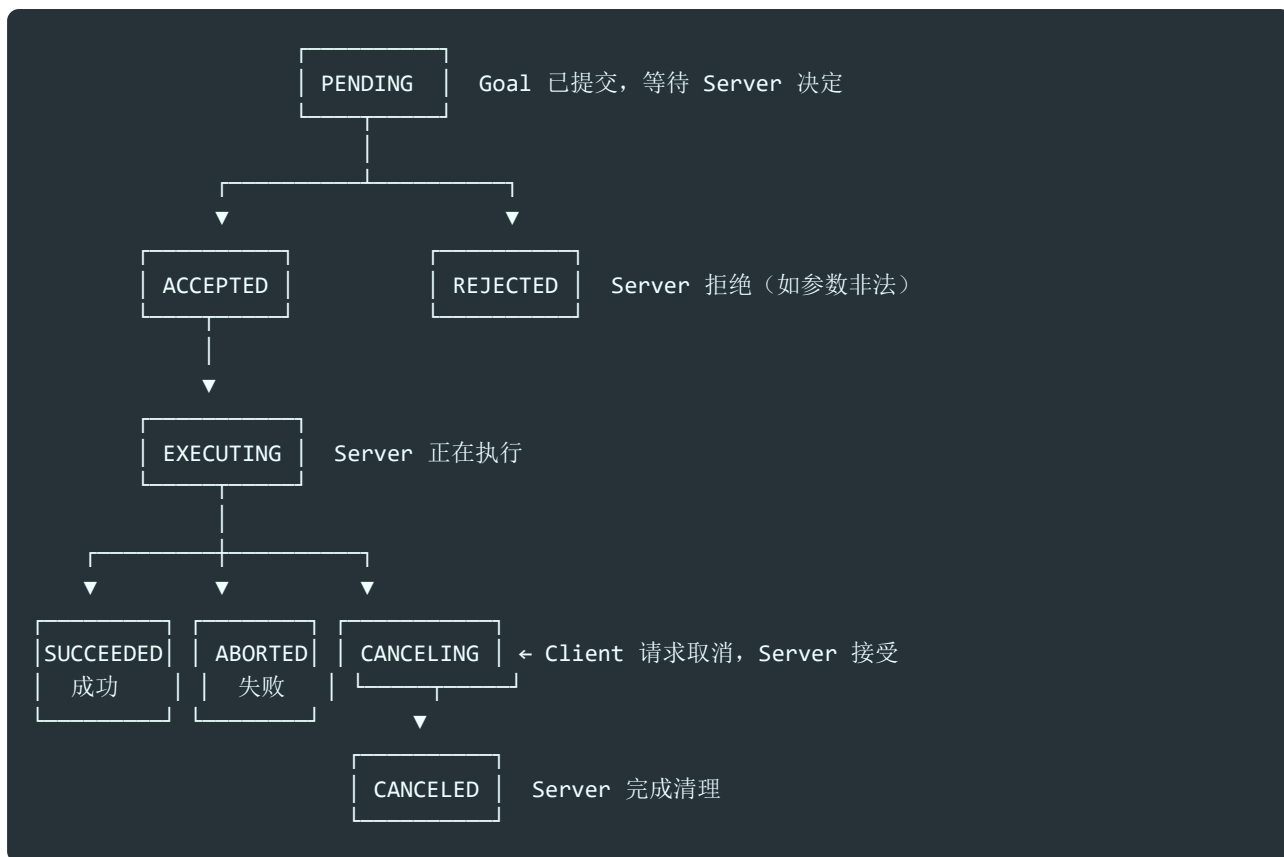


底层组件	作用	对应通信机制
<code>send_goal</code> Service	Client 向 Server 提交 Goal	Service
<code>cancel_goal</code> Service	Client 请求取消正在执行的任务	Service
<code>feedback</code> Topic	Server 持续发布进度信息	Topic
<code>status</code> Topic	自动发布所有 Goal 的状态变化	Topic
<code>get_result</code> Service	Client 请求获取 Goal 的最终结果	Service

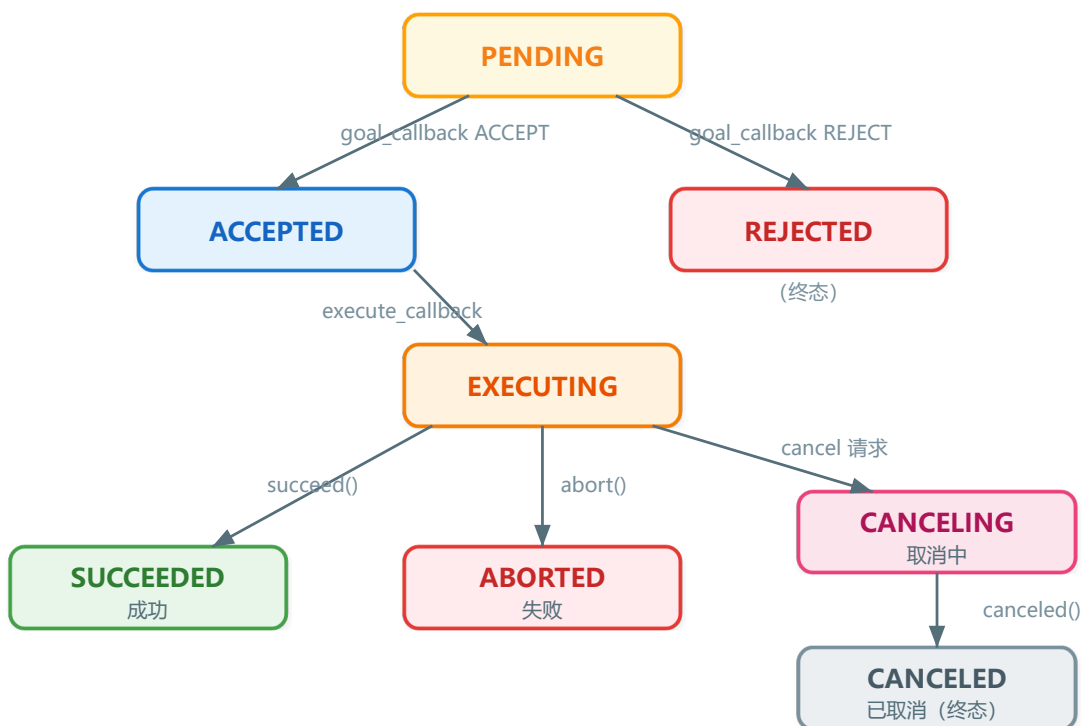
作为开发者，你**不需要直接操作这些底层 Service 和 Topic**——Python 侧的 `rclpy.action` 和 C++ 侧的 `rclcpp_action` 封装了全部细节，你只需关注 Goal、Feedback、Result 三个逻辑部分。

Goal 的生命周期

每一个被发送的 Goal 都经历明确的状态转换：



Goal 生命周期状态转换



状态	含义	触发条件
PENDING	等待处理	Client 调用 <code>send_goal</code> 后
ACCEPTED	已接受	Server 的 <code>goal_callback</code> 返回 ACCEPT
REJECTED	已拒绝	Server 的 <code>goal_callback</code> 返回 REJECT
EXECUTING	执行中	Server 开始 <code>execute_callback</code>
CANCELING	取消中	Client 请求取消, <code>cancel_callback</code> 返回 ACCEPT
SUCCEEDED	成功完成	Server 调用 <code>succeed()</code>
ABORTED	执行失败	Server 调用 <code>abort()</code> (如遇到错误)
CANCELED	已取消	Server 调用 <code>canceled()</code> (清理完成后)

注意 CANCELING 状态：取消不是瞬间完成的。Client 发起取消请求 → `cancel_callback` 接受 → Goal 进入 CANCELING → `execute_callback` 检测到 `is_cancel_requested` → 完成清理 → 调用 `canceled()` → 最终变为 CANCELED。

三种通信机制对比

维度	Topic	Service	Action
模式	发布/订阅	请求/响应	目标/反馈/结果
方向	单向	双向 (同步)	双向 (异步)
持续时间	持续流	瞬间完成	长时间任务
返回值	无	有 (Response)	有 (Result)
进度反馈	无 (消息本身就是数据)	无	有 (Feedback)
可取消	不适用	困难	内置支持
Client 阻塞	不阻塞	阻塞等待	不阻塞 (异步回调)
典型场景	传感器数据、速度指令	查询状态、触发拍照	导航、抓取、计数

选择通信机制的决策树：

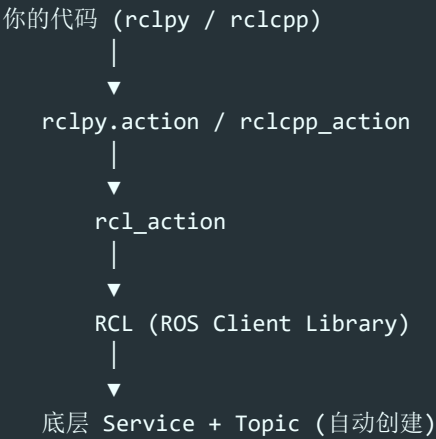
需要返回值吗？

- 否 → 需要持续发送数据吗？
 - 是 → Topic
 - 否 → 重新考虑设计
- 是 → 任务需要较长时间（>1秒）吗？
 - 是 → 需要中途反馈进度吗？
 - 是 → Action ✓
 - 否 → 仍可用 Action，或考虑 Service
 - 否 → Service ✓

典型场景速查：

场景	推荐机制	原因
激光雷达发布扫描数据	Topic	持续数据流，无需返回值
查询电池电量	Service	瞬间完成，需要返回值
导航到目标点	Action	耗时数分钟，需要进度反馈和取消
机械臂移动到指定位姿	Action	耗时数秒，需要反馈当前位置
从 1 数到 100（带间隔）	Action	耗时任务，需要反馈当前数字
触发一次拍照	Service	瞬间完成

Action 在 ROS 2 架构中的位置



Action API 位于 RCL 之上。Python 侧通过 `rclpy.action` 导入，依赖仍声明为 `rclpy`；C++ 侧使用独立的 `rclcpp_action` 包。使用 Action 时，节点除了是 `Node` 的子类，还需要创建 `ActionServer` 或 `ActionClient`。

内置 Action 示例

ROS 2 和常用功能包已内置多个 Action 定义：

Action 类型	所属包	用途
turtlesim/action/RotateAbsolute	turtlesim	海龟旋转到指定角度
nav2_msgs/action/NavigateToPose	nav2	导航到目标位姿
control_msgs/action/FollowJointTrajectory	control_msgs	关节轨迹跟踪
example_interfaces/action/Fibonacci	example_interfaces	官方教程示例

查看 Action 定义：

```
ros2 interface show turtlesim/action/RotateAbsolute

# The desired orientation in radians
float32 theta
---
# The angular displacement in radians to the starting position
float32 delta
---
# The remaining rotation in radians
float32 remaining
```

格式说明： --- 分隔 Goal、Result、Feedback 三个部分（与 .action 文件格式一致）。

关键点

- Action = **Goal (目标)** + **Feedback (反馈)** + **Result (结果)**，专为长时间任务设计
 - 底层由 **3 个 Service** (send_goal / cancel_goal / get_result) + **2 个 Topic** (feedback / status) 自动组成，开发者使用高层 API 即可
 - Goal 生命周期：**PENDING** → **ACCEPTED** → **EXECUTING** → **SUCCEEDED / ABORTED / (CANCELING → CANCELED)**
 - 与 Topic (持续流) 和 Service (瞬间响应) 互补，三者覆盖机器人通信的全部场景
 - Action Client 异步工作，通过回调函数接收 Feedback 和 Result，不会阻塞主线程
-

讲次 2：创建 Action 定义文件

知识点

Action 接口文件格式

Action 接口使用 `.action` 文件定义，语法与 `.msg` / `.srv` 类似，但包含三个部分，用 `---` 分隔：

```
# Goal 定义（上方）
字段类型 字段名
字段类型 字段名
---
# Result 定义（中间）
字段类型 字段名
---
# Feedback 定义（下方）
字段类型 字段名
```

本课程统一使用 `CountUntil.action`：

```
# Goal: 从 1 数到 target_number，每次间隔 period 秒
int64 target_number
float32 period
---
# Result: 最终到达的数字
int64 reached_number
---
# Feedback: 当前数到的数字
int64 current_number
```

创建接口功能包

Action 接口必须放在 `ament_cmake` 类型的接口包中（与自定义 `.msg` / `.srv` 相同）：

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake count_until_interfaces \
  --dependencies action_msgs
```

创建 Action 文件目录和定义：

```
mkdir -p ~/ros2_ws/src/count_until_interfaces/action
```

创建文件 `~/ros2_ws/src/count_until_interfaces/action/CountUntil.action`，内容如上。

生成的目录结构：

```
count_until_interfaces/  
├─ CMakeLists.txt  
├─ package.xml  
└─ action/  
    └─ CountUntil.action
```

配置 package.xml

编辑 `~/ros2_ws/src/count_until_interfaces/package.xml`，确保包含以下依赖：

```
<buildtool_depend>ament_cmake</buildtool_depend>  
<buildtool_depend>rosidl_default_generators</buildtool_depend>  
  
<depend>action_msgs</depend>  
  
<exec_depend>rosidl_default_runtime</exec_depend>  
  
<member_of_group>rosidl_interface_packages</member_of_group>
```

关键项说明：

依赖	作用
<code>rosidl_default_generators</code>	编译期生成 Action 的 Python/C++ 代码
<code>rosidl_default_runtime</code>	运行时使用生成的接口
<code>action_msgs</code>	Action 元信息类型（Goal 状态枚举等）
<code>member_of_group: rosidl_interface_packages</code>	声明本包是接口包

配置 CMakeLists.txt

编辑 `~/ros2_ws/src/count_until_interfaces/CMakeLists.txt`：

```

cmake_minimum_required(VERSION 3.8)
project(count_until_interfaces)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)
find_package(action_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/CountUntil.action"
  DEPENDENCIES action_msgs
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()

```

核心语句:

```

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/CountUntil.action"
  DEPENDENCIES action_msgs
)

```

`rosidl_generate_interfaces` 会读取 `.action` 文件, 自动生成 C++ 头文件和 Python 模块。

构建接口包

```

cd ~/ros2_ws
colcon build --packages-select count_until_interfaces
source install/setup.bash

```

构建成功后, 验证接口已注册:

```
ros2 interface list | grep count_until
```

预期输出:

```
count_until_interfaces/action/CountUntil
```

查看完整定义:

```
ros2 interface show count_until_interfaces/action/CountUntil
```

```
int64 target_number
float32 period
---
int64 reached_number
---
int64 current_number
```

生成的代码结构

编译后，ROS 2 自动生成以下类型（命名规则与 Message / Service 一致）：

Python 导入：

```
from count_until_interfaces.action import CountUntil

goal = CountUntil.Goal()
goal.target_number = 10
goal.period = 0.5

feedback = CountUntil.Feedback()
feedback.current_number = 5

result = CountUntil.Result()
result.reached_number = 10
```

C++ 导入：

```
#include "count_until_interfaces/action/count_until.hpp"

using CountUntil = count_until_interfaces::action::CountUntil;

auto goal = CountUntil::Goal();
goal.target_number = 10;
goal.period = 0.5f;
```

类型命名对照：

逻辑部分	Python 类	C++ 类型
完整 Action	CountUntil	count_until_interfaces::action::CountUntil
Goal	CountUntil.Goal	CountUntil::Goal
Feedback	CountUntil.Feedback	CountUntil::Feedback
Result	CountUntil.Result	CountUntil::Result

在其他功能包中声明依赖

编写 Action Server / Client 的功能包必须在 `package.xml` 中依赖接口包：

```
<!-- Python 功能包 (package.xml) -->
<depend>count_until_interfaces</depend>
<depend>rclpy</depend>
<depend>action_msgs</depend>
```

```
<!-- C++ 功能包 (package.xml) -->
<depend>count_until_interfaces</depend>
<depend>rclcpp</depend>
<depend>rclcpp_action</depend>
```

C++ 功能包还需在 `CMakeLists.txt` 中添加：

```
find_package(count_until_interfaces REQUIRED)
find_package(rclcpp_action REQUIRED)

# 方式 1（推荐）：使用 ament_target_dependencies 自动链接
ament_target_dependencies(server_node
  rclcpp rclcpp_action count_until_interfaces)

# 方式 2：手动链接（注意包名必须是接口包名，不是 ${PROJECT_NAME}）
# rosidl_get_typesupport_target(
#   cpp_typesupport_target count_until_interfaces "rosidl_typesupport_cpp")
# target_link_libraries(server_node ${cpp_typesupport_target})
```

常见错误： `rosidl_get_typesupport_target` 的第二个参数必须是**接口包名**（如 `count_until_interfaces`），而不是当前包名 `${PROJECT_NAME}`。写错会导致 `undefined reference` 编译错误。推荐直接用 `ament_target_dependencies` 更简洁。

关键点

- `.action` 文件用 `---` 分隔 Goal、Result、Feedback 三部分
 - Action 接口必须放在 `ament_cmake` 接口包中，使用 `rosidl_generate_interfaces` 生成代码
 - 完整类型写法：`count_until_interfaces/action/CountUntil`
 - 其他功能包通过 `package.xml` 声明依赖，C++ 还需 `target_link_libraries` 链接接口库
 - 用 `ros2 interface show` 验证 Action 定义是否正确注册
-

讲次 3：【PY】编写 Action Server

知识点

创建 Python 功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python py_action_tutorial \
  --dependencies rclpy action_msgs count_until_interfaces
```

Action Server 核心 API

API / 回调	作用
<code>ActionServer(node, ActionType, action_name, execute_callback, ...)</code>	创建 Action Server
<code>goal_callback(goal_request)</code>	决定是否接受 Goal (返回 ACCEPT / REJECT)
<code>execute_callback(goal_handle)</code>	执行任务, 发布 Feedback, 返回 Result
<code>cancel_callback(goal_handle)</code>	决定是否接受取消请求 (返回 ACCEPT / REJECT)
<code>goal_handle.publish_feedback(feedback)</code>	发布进度反馈
<code>goal_handle.is_cancel_requested</code>	检查是否收到取消请求
<code>goal_handle.succeed()</code>	标记 Goal 成功
<code>goal_handle.abort()</code>	标记 Goal 失败
<code>goal_handle.canceled()</code>	标记 Goal 已取消

为什么需要多线程执行器

`execute_callback` 通常包含循环和 `sleep`, 是**阻塞操作**。如果与 `goal_callback` 共用单线程执行器, 执行期间将无法处理新的 Goal 请求和取消请求。

解决方案: 使用 `MultiThreadedExecutor` + `ReentrantCallbackGroup`。

完整代码: Action Server 节点

创建文件 `~/ros2_ws/src/py_action_tutorial/py_action_tutorial/count_until_server.py` :

```

#!/usr/bin/env python3
import time

import rclpy
from rclpy.action import ActionServer, CancelResponse, GoalResponse
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import ExternalShutdownException, MultiThreadedExecutor
from rclpy.node import Node

from count_until_interfaces.action import CountUntil

class CountUntilServer(Node):
    """从 1 数到 target_number, 持续发布 Feedback, 支持取消。"""

    def __init__(self):
        super().__init__("count_until_server")
        self._action_server = ActionServer(
            self,
            CountUntil,
            "count_until",
            self.execute_callback,
            callback_group=ReentrantCallbackGroup(),
            goal_callback=self.goal_callback,
            cancel_callback=self.cancel_callback,
        )
        self.get_logger().info("Action Server ready on /count_until")

    def goal_callback(self, goal_request):
        """校验 Goal 参数, 决定接受或拒绝。"""
        self.get_logger().info(
            f"Received goal: target={goal_request.target_number}, "
            f"period={goal_request.period}"
        )
        if goal_request.target_number <= 0:
            self.get_logger().warn("Rejecting goal: target_number must be positive")
            return GoalResponse.REJECT
        if goal_request.period <= 0.0:
            self.get_logger().warn("Rejecting goal: period must be positive")
            return GoalResponse.REJECT
        return GoalResponse.ACCEPT

    def cancel_callback(self, goal_handle):
        """接受取消请求, execute_callback 中会检测并响应。"""
        self.get_logger().info("Received cancel request, accepting")
        return CancelResponse.ACCEPT

    def execute_callback(self, goal_handle):
        """执行计数任务, 发布 Feedback, 检查取消, 返回 Result。"""
        target = goal_handle.request.target_number
        period = goal_handle.request.period
        feedback_msg = CountUntil.Feedback()

        self.get_logger().info(f"Executing: counting to {target}")

        for i in range(1, target + 1):
            if goal_handle.is_cancel_requested:

```

```

        goal_handle.canceled()
        self.get_logger().info(f"Goal canceled at {i - 1}")
        result = CountUntil.Result()
        result.reached_number = i - 1
        return result

    feedback_msg.current_number = i
    goal_handle.publish_feedback(feedback_msg)
    self.get_logger().info(f"Feedback: {i}/{target}")
    time.sleep(period)

    result = CountUntil.Result()
    result.reached_number = target
    goal_handle.succeed()
    self.get_logger().info(f"Goal succeeded: reached {target}")
    return result

def main(args=None):
    rclpy.init(args=args)
    node = CountUntilServer()
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    try:
        executor.spin()
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        executor.shutdown(timeout_sec=2.0)
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    main()

```

代码解析

1. 创建 ActionServer

```

self._action_server = ActionServer(
    self,
    CountUntil,
    "count_until",
    self.execute_callback,
    callback_group=ReentrantCallbackGroup(),
    goal_callback=self.goal_callback,
    cancel_callback=self.cancel_callback,
)

```

- 第一个参数：节点自身（ `self` ）
- 第二个参数：Action 类型
- 第三个参数：Action 名称（对应 `/count_until` ）

- `callback_group=ReentrantCallbackGroup()`：允许回调并行执行，避免 `execute` 阻塞其他回调

2. goal_callback — Accept / Reject

```
def goal_callback(self, goal_request):
    if goal_request.target_number <= 0:
        return GoalResponse.REJECT
    return GoalResponse.ACCEPT
```

- 在 Goal 进入 EXECUTING 状态之前调用
- 用于参数校验、资源检查（如"机械臂是否空闲"）
- 返回 `GoalResponse.ACCEPT` 或 `GoalResponse.REJECT`

3. execute_callback — 核心执行逻辑

```
def execute_callback(self, goal_handle):
    for i in range(1, target + 1):
        if goal_handle.is_cancel_requested:
            goal_handle.canceled()
            return result
        goal_handle.publish_feedback(feedback_msg)
        time.sleep(period)
    goal_handle.succeed()
    return result
```

- `goal_handle.request` 获取 Goal 字段
- 循环中定期检查 `is_cancel_requested`
- 调用 `publish_feedback()` 发布进度
- 结束时调用 `succeed()` / `abort()` / `canceled()` 设置最终状态
- 返回值是 `Result` 消息

4. cancel_callback

```
def cancel_callback(self, goal_handle):
    return CancelResponse.ACCEPT
```

- 决定是否接受取消请求（不是立即终止任务）
- 返回 ACCEPT 后，`execute_callback` 中的 `is_cancel_requested` 会变为 `True`
- 实际清理工作仍在 `execute_callback` 中完成

5. Succeed vs Abort vs Canceled

方法	状态	使用场景
<code>goal_handle.succeed()</code>	SUCCEEDED	任务正常完成
<code>goal_handle.abort()</code>	ABORTED	遇到错误无法完成（如传感器故障）
<code>goal_handle.canceled()</code>	CANCELED	用户主动取消

Abort 示例（可在 `execute_callback` 中加入）：

```
if some_error_condition:
    result = CountUntil.Result()
    result.reached_number = i
    goal_handle.abort()
    self.get_logger().error("Goal aborted due to error")
    return result
```

注册入口点并运行

编辑 `setup.py`：

```
entry_points={
    'console_scripts': [
        'count_until_server = py_action_tutorial.count_until_server:main',
    ],
},
```

构建并运行：

```
cd ~/ros2_ws
colcon build --packages-select py_action_tutorial
source install/setup.bash
ros2 run py_action_tutorial count_until_server
```

验证 Server 已启动：

```
ros2 action list -t
```

预期输出包含：

```
/count_until [count_until_interfaces/action/CountUntil]
```

关键点

- Action Server 有三个核心回调：`goal_callback`（接受/拒绝）、`execute_callback`（执行）、`cancel_callback`（取消）
 - `execute_callback` 是阻塞的，必须使用 `MultiThreadedExecutor` + `ReentrantCallbackGroup`
 - 在循环中检查 `is_cancel_requested`，实现优雅取消
 - 任务结束时必须调用 `succeed()` / `abort()` / `canceled()` 之一，并返回 `Result`
 - `publish_feedback()` 可在执行过程中任意调用，Client 通过回调实时接收
-

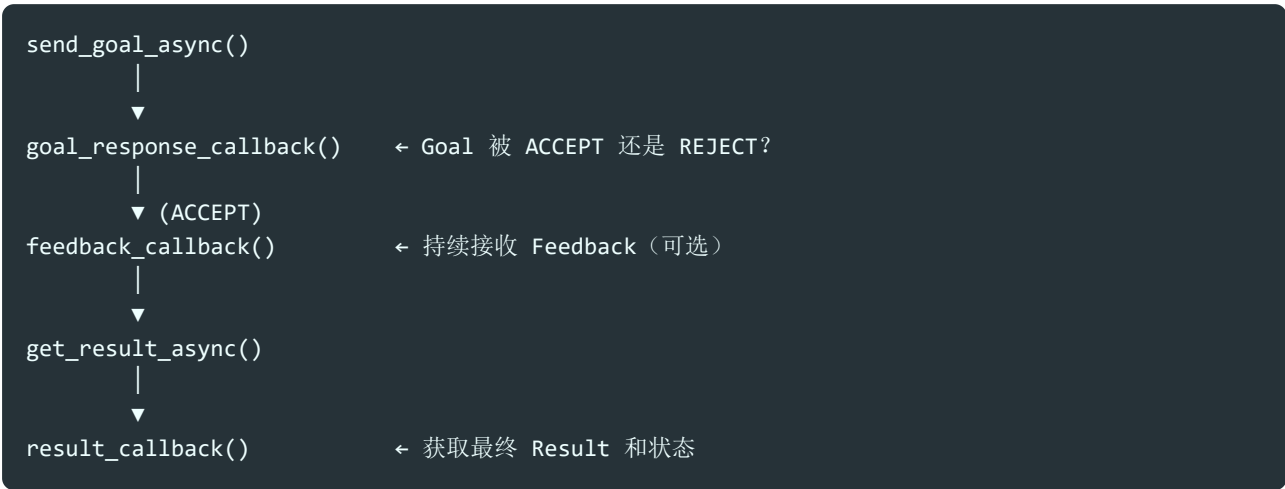
讲次 4：【PY】编写 Action Client

知识点

Action Client 核心 API

API / 回调	作用
<code>ActionClient(node, ActionType, action_name)</code>	创建 Action Client
<code>wait_for_server(timeout_sec=...)</code>	等待 Server 可用
<code>send_goal_async(goal, feedback_callback=...)</code>	异步发送 Goal
<code>feedback_callback(feedback_msg)</code>	接收 Feedback 的回调
<code>goal_handle.get_result_async()</code>	异步获取 Result
<code>goal_handle.cancel_goal_async()</code>	异步请求取消 Goal

Action Client 的工作流程：



完整代码：Action Client 节点

创建文件 `~/ros2_ws/src/py_action_tutorial/py_action_tutorial/count_until_client.py`：

```

#!/usr/bin/env python3
import rclpy
from action_msgs.msg import GoalStatus
from rclpy.action import ActionClient
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node

from count_until_interfaces.action import CountUntil

class CountUntilClient(Node):
    """向 CountUntil Action Server 发送 Goal, 接收 Feedback 和 Result."""

    def __init__(self):
        super().__init__("count_until_client")
        self._action_client = ActionClient(self, CountUntil, "count_until")

    def send_goal(self, target_number, period):
        self.get_logger().info("Waiting for action server...")
        if not self._action_client.wait_for_server(timeout_sec=5.0):
            self.get_logger().error("Action server not available")
            rclpy.shutdown()
            return

        goal_msg = CountUntil.Goal()
        goal_msg.target_number = target_number
        goal_msg.period = period

        self.get_logger().info(
            f"Sending goal: target={target_number}, period={period}"
        )

        send_goal_future = self._action_client.send_goal_async(
            goal_msg,
            feedback_callback=self.feedback_callback,
        )
        send_goal_future.add_done_callback(self.goal_response_callback)

    def goal_response_callback(self, future):
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().warn("Goal was rejected by server")
            rclpy.shutdown()
            return

        self.get_logger().info("Goal accepted, waiting for result...")
        get_result_future = goal_handle.get_result_async()
        get_result_future.add_done_callback(self.get_result_callback)

    def feedback_callback(self, feedback_msg):
        feedback = feedback_msg.feedback
        self.get_logger().info(f"Feedback: current_number={feedback.current_number}")

    def get_result_callback(self, future):
        wrapped_result = future.result()
        result = wrapped_result.result
        status_name = {

```

```

        GoalStatus.STATUS_SUCCEEDED: "SUCCEEDED",
        GoalStatus.STATUS_CANCELED: "CANCELED",
        GoalStatus.STATUS_ABORTED: "ABORTED",
    }.get(wrapped_result.status, str(wrapped_result.status))
    self.get_logger().info(
        f"Result: reached_number={result.reached_number}, status={status_name}"
    )
    rclpy.shutdown()

def main(args=None):
    rclpy.init(args=args)
    node = CountUntilClient()
    node.send_goal(target_number=10, period=0.5)
    try:
        rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    main()

```

代码解析

1. 创建 ActionClient 并等待 Server

```

self._action_client = ActionClient(self, CountUntil, "count_until")
self._action_client.wait_for_server()

```

`wait_for_server()` 阻塞直到 Server 可用。生产代码中建议设置超时：

```

if not self._action_client.wait_for_server(timeout_sec=5.0):
    self.get_logger().error("Action server not available")
    return

```

2. 构造 Goal 并异步发送

```

goal_msg = CountUntil.Goal()
goal_msg.target_number = target_number
goal_msg.period = period

send_goal_future = self._action_client.send_goal_async(
    goal_msg,
    feedback_callback=self.feedback_callback,
)
send_goal_future.add_done_callback(self.goal_response_callback)

```

- `send_goal_async` 立即返回 Future, 不阻塞
- `feedback_callback` 在 Goal 执行期间被反复调用
- `goal_response_callback` 在 Server 接受/拒绝后触发

3. 处理 Goal 响应

```
def goal_response_callback(self, future):
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().warn("Goal was rejected by server")
        return
    get_result_future = goal_handle.get_result_async()
    get_result_future.add_done_callback(self.get_result_callback)
```

- `goal_handle.accepted` 检查 Server 是否接受了 Goal
- 接受后, 通过 `get_result_async()` 注册 Result 回调

4. Feedback 回调

```
def feedback_callback(self, feedback_msg):
    feedback = feedback_msg.feedback
    self.get_logger().info(f"Feedback: current_number={feedback.current_number}")
```

- 每收到一条 Feedback 消息, 此回调被调用一次
- `feedback_msg.feedback` 是 `CountUntil.Feedback` 类型

5. Result 回调

```
def get_result_callback(self, future):
    wrapped_result = future.result()
    result = wrapped_result.result
    status = wrapped_result.status
```

- `future.result().result` 是 `CountUntil.Result` 类型
- `future.result().status` 是 Goal 最终状态; 业务代码应与 `GoalStatus` 常量比较, 不要直接写数字

注册入口点

编辑 `setup.py` :

```
entry_points={
    'console_scripts': [
        'count_until_server = py_action_tutorial.count_until_server:main',
        'count_until_client = py_action_tutorial.count_until_client:main',
    ],
}
```

运行测试

终端 1 — 启动 Server:

```
ros2 run py_action_tutorial count_until_server
```

终端 2 — 启动 Client:

```
ros2 run py_action_tutorial count_until_client
```

预期输出 (终端 2) :

```
[INFO] Waiting for action server...
[INFO] Sending goal: target=10, period=0.5
[INFO] Goal accepted, waiting for result...
[INFO] Feedback: current_number=1
[INFO] Feedback: current_number=2
...
[INFO] Feedback: current_number=10
[INFO] Result: reached_number=10, status=SUCCEEDED
```

发送取消请求

扩展 Client, 支持在 Feedback 达到 5 时自动取消:

```
def feedback_callback(self, feedback_msg):
    feedback = feedback_msg.feedback
    self.get_logger().info(f"Feedback: current_number={feedback.current_number}")
    if feedback.current_number >= 5 and not self._cancel_sent:
        self.get_logger().info("Canceling goal at 5...")
        self._goal_handle.cancel_goal_async()
        self._cancel_sent = True
```

在 `goal_response_callback` 中保存 `goal_handle` :

```
self._goal_handle = goal_handle
self._cancel_sent = False
```

关键点

- Action Client 通过 `send_goal_async` 异步发送 Goal, 不阻塞主线程
 - 回调链: `goal_response_callback` → `feedback_callback` (多次) → `get_result_callback`
 - `wait_for_server()` 确保 Server 可用后再发送 Goal
 - 通过 `goal_handle.cancel_goal_async()` 可随时请求取消
 - `future.result().status` 可判断最终状态 (成功/取消/失败)
-

讲次 5：【C++】编写 Action Server

知识点

创建 C++ 功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake cpp_action_tutorial \
  --dependencies rclcpp rclcpp_action count_until_interfaces
```

C++ Action Server 核心 API

API / 回调	作用
<code>rclcpp_action::create_server<ActionT>(...)</code>	创建 Action Server
<code>handle_goal(uuid, goal)</code>	返回 <code>GoalResponse::ACCEPT_AND_EXECUTE</code> 、 <code>ACCEPT_AND_DEFER</code> 或 <code>REJECT</code>
<code>handle_cancel(goal_handle)</code>	返回 <code>CancelResponse::ACCEPT</code> 或 <code>REJECT</code>
<code>handle_accepted(goal_handle)</code>	Goal 被接受后的入口，通常启动新线程执行
<code>goal_handle->publish_feedback(feedback)</code>	发布 Feedback
<code>goal_handle->is_canceling()</code>	检查是否正在取消
<code>goal_handle->succeed(result)</code>	标记成功
<code>goal_handle->abort(result)</code>	标记失败
<code>goal_handle->canceled(result)</code>	标记已取消

C++ 与 Python 的重要区别：`execute_callback` 不是直接传给 `create_server`，而是在 `handle_accepted` 中手动启动新线程执行，避免阻塞执行器。

完整代码：C++ Action Server

创建文件 `~/ros2_ws/src/cpp_action_tutorial/src/count_until_server.cpp`：


```

#include <chrono>
#include <memory>
#include <thread>

#include "count_until_interfaces/action/count_until.hpp"
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"

using CountUntil = count_until_interfaces::action::CountUntil;
using GoalHandleCountUntil = rclcpp_action::ServerGoalHandle<CountUntil>;

class CountUntilActionServer : public rclcpp::Node
{
public:
    CountUntilActionServer()
    : Node("count_until_server")
    {
        action_server_ = rclcpp_action::create_server<CountUntil>(
            this,
            "count_until",
            std::bind(&CountUntilActionServer::handle_goal, this, std::placeholders::_1, std::placeholders::_2),
            std::bind(&CountUntilActionServer::handle_cancel, this, std::placeholders::_1),
            std::bind(&CountUntilActionServer::handle_accepted, this, std::placeholders::_1));

        RCLCPP_INFO(this->get_logger(), "Action Server ready on /count_until");
    }

private:
    rclcpp_action::GoalResponse handle_goal(
        const rclcpp_action::GoalUUID & /*uuid*/,
        std::shared_ptr<const CountUntil::Goal> goal)
    {
        RCLCPP_INFO(
            this->get_logger(),
            "Received goal: target=%ld, period=%.2f",
            goal->target_number, goal->period);

        if (goal->target_number <= 0) {
            RCLCPP_WARN(this->get_logger(), "Rejecting goal: target_number must be positive");
            return rclcpp_action::GoalResponse::REJECT;
        }
        if (goal->period <= 0.0f) {
            RCLCPP_WARN(this->get_logger(), "Rejecting goal: period must be positive");
            return rclcpp_action::GoalResponse::REJECT;
        }
        return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE;
    }

    rclcpp_action::CancelResponse handle_cancel(
        const std::shared_ptr<GoalHandleCountUntil> /*goal_handle*/)
    {
        RCLCPP_INFO(this->get_logger(), "Received cancel request, accepting");
        return rclcpp_action::CancelResponse::ACCEPT;
    }

    void handle_accepted(const std::shared_ptr<GoalHandleCountUntil> goal_handle)
    {

```

```

    std::thread{std::bind(&CountUntilActionServer::execute, this, std::placeholders::_1), goal_handle}
    .detach();
}

void execute(const std::shared_ptr<GoalHandleCountUntil> goal_handle)
{
    const auto goal = goal_handle->get_goal();
    auto feedback = std::make_shared<CountUntil::Feedback>();
    auto result = std::make_shared<CountUntil::Result>();

    RCLCPP_INFO(this->get_logger(), "Executing: counting to %ld", goal->target_number);

    rclcpp::Rate loop_rate(1.0 / goal->period);

    for (int64_t i = 1; i <= goal->target_number; ++i) {
        if (goal_handle->is_canceling()) {
            result->reached_number = i - 1;
            goal_handle->cancel(result);
            RCLCPP_INFO(this->get_logger(), "Goal canceled at %ld", i - 1);
            return;
        }

        feedback->current_number = i;
        goal_handle->publish_feedback(feedback);
        RCLCPP_INFO(this->get_logger(), "Feedback: %ld/%ld", i, goal->target_number);
        loop_rate.sleep();
    }

    result->reached_number = goal->target_number;
    goal_handle->succeed(result);
    RCLCPP_INFO(this->get_logger(), "Goal succeeded: reached %ld", goal->target_number);
}

rclcpp_action::Server<CountUntil>::SharedPtr action_server_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<CountUntilActionServer>();
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}

```

代码解析

1. 类型别名

```

using CountUntil = count_until_interfaces::action::CountUntil;
using GoalHandleCountUntil = rclcpp_action::ServerGoalHandle<CountUntil>;

```

`ServerGoalHandle` 是操作 Goal 的核心句柄，贯穿 `handle_cancel`、`handle_accepted` 和 `execute`。

2. `create_server` 的三个回调

```
rclcpp_action::create_server<CountUntil>(
    this, "count_until",
    handle_goal,      // 决定是否接受
    handle_cancel,    // 决定是否允许取消
    handle_accepted   // 接受后启动执行
);
```

3. handle_accepted 中启动新线程

```
void handle_accepted(const std::shared_ptr<GoalHandleCountUntil> goal_handle)
{
    std::thread{std::bind(&CountUntilActionServer::execute, this, _1), goal_handle}.detach();
}
```

这是教程中常见的一种模式：让 `execute` 在独立线程运行，避免阻塞 `rclcpp::spin()`。它不是唯一实现；使用 `detach()` 时必须保证节点和 Goal Handle 在线程结束前仍有效，并设计好取消与关机收尾。生产代码也可采用受管理线程、任务队列或合适的 Executor/Callback Group。

4. execute 中的取消检查

```
if (goal_handle->is_canceling()) {
    result->reached_number = i - 1;
    goal_handle->canceled(result);
    return;
}
```

注意 C++ 使用 `is_canceling()`（而非 Python 的 `is_cancel_requested`）。

5. 结束状态设置

```
goal_handle->succeed(result); // 成功
goal_handle->abort(result);   // 失败
goal_handle->canceled(result); // 取消
```

C++ 中将 Result 作为参数传入；Python 中是先设置状态再 return Result。

配置 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(cpp_action_tutorial)

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_action REQUIRED)
find_package(count_until_interfaces REQUIRED)

add_executable(count_until_server src/count_until_server.cpp)

ament_target_dependencies(count_until_server
  rclcpp rclcpp_action count_until_interfaces)

install(TARGETS count_until_server
  DESTINATION lib/${PROJECT_NAME})

ament_package()
```

构建并运行：

```
cd ~/ros2_ws
colcon build --packages-select cpp_action_tutorial
source install/setup.bash
ros2 run cpp_action_tutorial count_until_server
```

关键点

- C++ Action Server 使用 `rclcpp_action::create_server<ActionT>()` 创建
 - 长时间 `execute` 不应阻塞负责接收 Goal/Cancel 的执行路径；本例在 `handle_accepted` 中启动线程，实际项目还需管理线程生命周期
 - `ServerGoalHandle` 是操作 Goal 的核心句柄
 - 取消检查使用 `is_canceling()`，结束后调用 `succeed()` / `abort()` / `canceled()`
 - `rclcpp::Rate` 可替代 `sleep`，精确控制 Feedback 发布频率
-

讲次 6: 【C++】编写 Action Client

知识点

C++ Action Client 核心 API

API / 回调	作用
<code>rclcpp_action::create_client<ActionT>(node, action_name)</code>	创建 Action Client
<code>client->wait_for_action_server(...)</code>	等待 Server 可用
<code>client->async_send_goal(goal, options)</code>	异步发送 Goal
<code>SendGoalOptions.feedback_callback</code>	Feedback 回调
<code>SendGoalOptions.result_callback</code>	Result 回调
<code>SendGoalOptions.goal_response_callback</code>	Goal 接受/拒绝回调
<code>client->async_cancel_goal(goal_handle)</code>	异步取消 Goal

完整代码: C++ Action Client

创建文件 `~/ros2_ws/src/cpp_action_tutorial/src/count_until_client.cpp` :

```

#include <chrono>
#include <functional>
#include <future>
#include <memory>

#include "count_until_interfaces/action/count_until.hpp"
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"

using CountUntil = count_until_interfaces::action::CountUntil;
using GoalHandleCountUntil = rclcpp_action::ClientGoalHandle<CountUntil>;

class CountUntilActionClient : public rclcpp::Node
{
public:
    CountUntilActionClient()
    : Node("count_until_client")
    {
        client_ptr_ = rclcpp_action::create_client<CountUntil>(this, "count_until");
    }

    void send_goal(int target_number, float period)
    {
        if (!client_ptr_->wait_for_action_server(std::chrono::seconds(5))) {
            RCLCPP_ERROR(this->get_logger(), "Action server not available");
            return;
        }

        auto goal_msg = CountUntil::Goal();
        goal_msg.target_number = target_number;
        goal_msg.period = period;

        RCLCPP_INFO(
            this->get_logger(),
            "Sending goal: target=%ld, period=%.2f",
            target_number, period);

        auto send_goal_options = rclcpp_action::Client<CountUntil>::SendGoalOptions();
        send_goal_options.feedback_callback =
            std::bind(&CountUntilActionClient::feedback_callback, this, std::placeholders::_1, std::placeholders::_2);
        send_goal_options.result_callback =
            std::bind(&CountUntilActionClient::result_callback, this, std::placeholders::_1);

        client_ptr_->async_send_goal(goal_msg, send_goal_options);
    }

private:
    void feedback_callback(
        GoalHandleCountUntil::SharedPtr,
        const std::shared_ptr<const CountUntil::Feedback> feedback)
    {
        RCLCPP_INFO(this->get_logger(), "Feedback: current_number=%ld", feedback->current_number);
    }

    void result_callback(const GoalHandleCountUntil::WrappedResult & result)
    {
        switch (result.code) {

```

```

        case rclcpp_action::ResultCode::SUCCEEDED:
            RCLCPP_INFO(
                this->get_logger(), "Goal succeeded: reached_number=%ld",
                result.result->reached_number);
            break;
        case rclcpp_action::ResultCode::ABORTED:
            RCLCPP_ERROR(this->get_logger(), "Goal aborted");
            break;
        case rclcpp_action::ResultCode::CANCELED:
            RCLCPP_WARN(this->get_logger(), "Goal canceled");
            break;
        default:
            RCLCPP_ERROR(this->get_logger(), "Unknown result code");
            break;
    }
    rclcpp::shutdown();
}

rclcpp_action::Client<CountUntil>::SharedPtr client_ptr_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<CountUntilActionClient>();
    node->send_goal(10, 0.5f);
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}

```

代码解析

1. Client 端 GoalHandle 类型

```
using GoalHandleCountUntil = rclcpp_action::ClientGoalHandle<CountUntil>;
```

Server 端用 `ServerGoalHandle` , Client 端用 `ClientGoalHandle` , 勿混淆。

2. SendGoalOptions 注册回调

```

auto send_goal_options = rclcpp_action::Client<CountUntil>::SendGoalOptions();
send_goal_options.feedback_callback = std::bind(...);
send_goal_options.result_callback = std::bind(...);
client_ptr_->async_send_goal(goal_msg, send_goal_options);

```

C++ 将三个回调统一放在 `SendGoalOptions` 中 (Python 是分开传递) 。

3. result_callback 与 WrappedResult

```
void result_callback(const GoalHandleCountUntil::WrappedResult & result)
{
    switch (result.code) {
        case rclcpp_action::ResultCode::SUCCEEDED: ...
        case rclcpp_action::ResultCode::ABORTED: ...
        case rclcpp_action::ResultCode::CANCELED: ...
    }
    result.result->reached_number; // 访问 Result 字段
}
```

`WrappedResult` 同时包含 `code` (状态码) 和 `result` (Result 消息)。

更新 CMakeLists.txt

```
add_executable(count_until_client src/count_until_client.cpp)
ament_target_dependencies(count_until_client
    rclcpp rclcpp_action count_until_interfaces)

install(TARGETS count_until_server count_until_client
    DESTINATION lib/${PROJECT_NAME})
```

运行测试

终端 1:

```
ros2 run cpp_action_tutorial count_until_server
```

终端 2:

```
ros2 run cpp_action_tutorial count_until_client
```

Python vs C++ API 对照

操作	Python	C++
创建 Server	<code>ActionServer(...)</code>	<code>rclcpp_action::create_server<>()</code>
创建 Client	<code>ActionClient(...)</code>	<code>rclcpp_action::create_client<>()</code>
发送 Goal	<code>send_goal_async()</code>	<code>async_send_goal()</code>
取消检查	<code>is_cancel_requested</code>	<code>is_canceling()</code>
标记成功	<code>goal_handle.succeed()</code>	<code>goal_handle->succeed(result)</code>
获取 Result	<code>future.result().result</code>	<code>result.result->field</code>

关键点

- C++ Client 使用 `SendGoalOptions` 统一注册 `feedback / result / goal_response` 回调
 - `WrappedResult` 包含 `code` 和 `result` , 通过 `ResultCode` 枚举判断最终状态
 - `wait_for_action_server()` 应设置超时, 避免无限等待
 - Client 端 `GoalHandle` 类型是 `ClientGoalHandle` , 与 Server 端不同
 - `async_send_goal` 是非阻塞的, 结果通过回调异步返回
-

讲次 7: 【LAB】Action Server / Client 综合实战

知识点

本实验将 Action 通信与 Turtlesim 仿真结合，实现“**海龟移动到指定位置**”的完整 Action 系统。相比 Turtlesim 内置的 `RotateAbsolute`（仅控制旋转），本实验同时控制线速度和角速度，是更接近真实导航的入门练习。

实验目标

1. 定义 `MoveToPosition.action` 接口
2. 编写 Action Server：读取海龟位姿，发布速度指令，持续反馈距离
3. 编写 Action Client：发送目标坐标，监控 Feedback，处理 Result
4. 在 Turtlesim 中验证完整流程，并测试取消功能

Step 1: 定义 MoveToPosition Action

在 `count_until_interfaces` 包中新增 Action 文件（或创建独立的 `turtle_action_interfaces` 包）：

创建 `~/ros2_ws/src/count_until_interfaces/action/MoveToPosition.action`：

```
# Goal: 目标位置
float32 target_x
float32 target_y
---
# Result: 最终位置与误差
float32 final_x
float32 final_y
float32 distance_error
---
# Feedback: 当前位置与剩余距离
float32 current_x
float32 current_y
float32 distance_remaining
```

更新 `CMakeLists.txt`：

```
rosidl_generate_interfaces(${PROJECT_NAME}
  "action/CountUntil.action"
  "action/MoveToPosition.action"
  DEPENDENCIES action_msgs
)
```

重新构建：

```
cd ~/ros2_ws
colcon build --packages-select count_until_interfaces
source install/setup.bash
```

Step 2: 编写 Action Server

在 `py_action_tutorial` 中新增 `move_to_position_server.py`。节点生命周期、`MultiThreadedExecutor` 和 `ReentrantCallbackGroup` 沿用讲次 3；本实验只新增机器人 I/O：

```
self.pose_: Pose | None = None
self.publisher_ = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
self.subscription_ = self.create_subscription(
    Pose,
    "/turtle1/pose",
    self.pose_callback,
    10,
    callback_group=self.callback_group_,
)

def pose_callback(self, pose: Pose):
    self.pose_ = pose
```

`execute_callback()` 的控制循环由四步组成：检查取消、计算距离、发布 Feedback、发布速度。核心逻辑如下：

```

while rclpy.ok():
    if goal_handle.is_cancel_requested:
        self.stop_turtle()
        goal_handle.canceled()
        return result

    if self.pose_ is None:
        # 最终源码设置 5 秒首帧超时，避免传感器缺失时永久等待
        time.sleep(0.05)
        continue

    dx = goal_handle.request.target_x - self.pose_.x
    dy = goal_handle.request.target_y - self.pose_.y
    distance = math.hypot(dx, dy)

    feedback.current_x = self.pose_.x
    feedback.current_y = self.pose_.y
    feedback.distance_remaining = distance
    goal_handle.publish_feedback(feedback)

    if distance < 0.15:
        self.stop_turtle()
        result.final_x = self.pose_.x
        result.final_y = self.pose_.y
        result.distance_error = distance
        goal_handle.succeed()
        return result

    angle_error = self.normalize_angle(math.atan2(dy, dx) - self.pose_.theta)
    cmd = Twist()
    cmd.linear.x = min(2.0, distance)
    cmd.angular.z = 6.0 * angle_error
    self.publisher_.publish(cmd)
    time.sleep(0.05)

```

可运行版本见 `stage2_code/py_action_tutorial/py_action_tutorial/move_to_position_server.py`。它还实现了目标范围校验、首帧 Pose 超时、同一机器人只接受一个活动 Goal，以及退出时零速度停车；这些工程边界不再在项目篇重复粘贴。

Step 3: 编写 Action Client

Client 的回调链与讲次 4 完全相同，只替换 Action 类型、Goal 字段和日志字段：

```

self.client_ = ActionClient(self, MoveToPosition, "move_to_position")

goal = MoveToPosition.Goal()
goal.target_x = float(x)
goal.target_y = float(y)
future = self.client_.send_goal_async(
    goal, feedback_callback=self.feedback_callback)
future.add_done_callback(self.goal_response_callback)

def feedback_callback(self, feedback_msg):
    feedback = feedback_msg.feedback
    self.get_logger().info(
        f"pose=({feedback.current_x:.2f}, {feedback.current_y:.2f}), "
        f"remaining={feedback.distance_remaining:.2f}")

def result_callback(self, future):
    wrapped_result = future.result()
    result = wrapped_result.result
    self.get_logger().info(
        f"final=({result.final_x:.2f}, {result.final_y:.2f}), "
        f"error={result.distance_error:.3f}, status={wrapped_result.status}")

```

发送前仍需 `wait_for_server(timeout_sec=5.0)`；Goal 请求和 Result Future 都要捕获异常，最终状态使用 `GoalStatus` 常量解释。完整版本见 `stage2_code/py_action_tutorial/py_action_tutorial/move_to_position_client.py`。

注册入口点（`setup.py`）：

```

'move_to_position_server' = py_action_tutorial.move_to_position_server:main',
'move_to_position_client' = py_action_tutorial.move_to_position_client:main',

```

Step 4: 运行实验

终端 1 — Turtlesim:

```
ros2 run turtlesim turtlesim_node
```

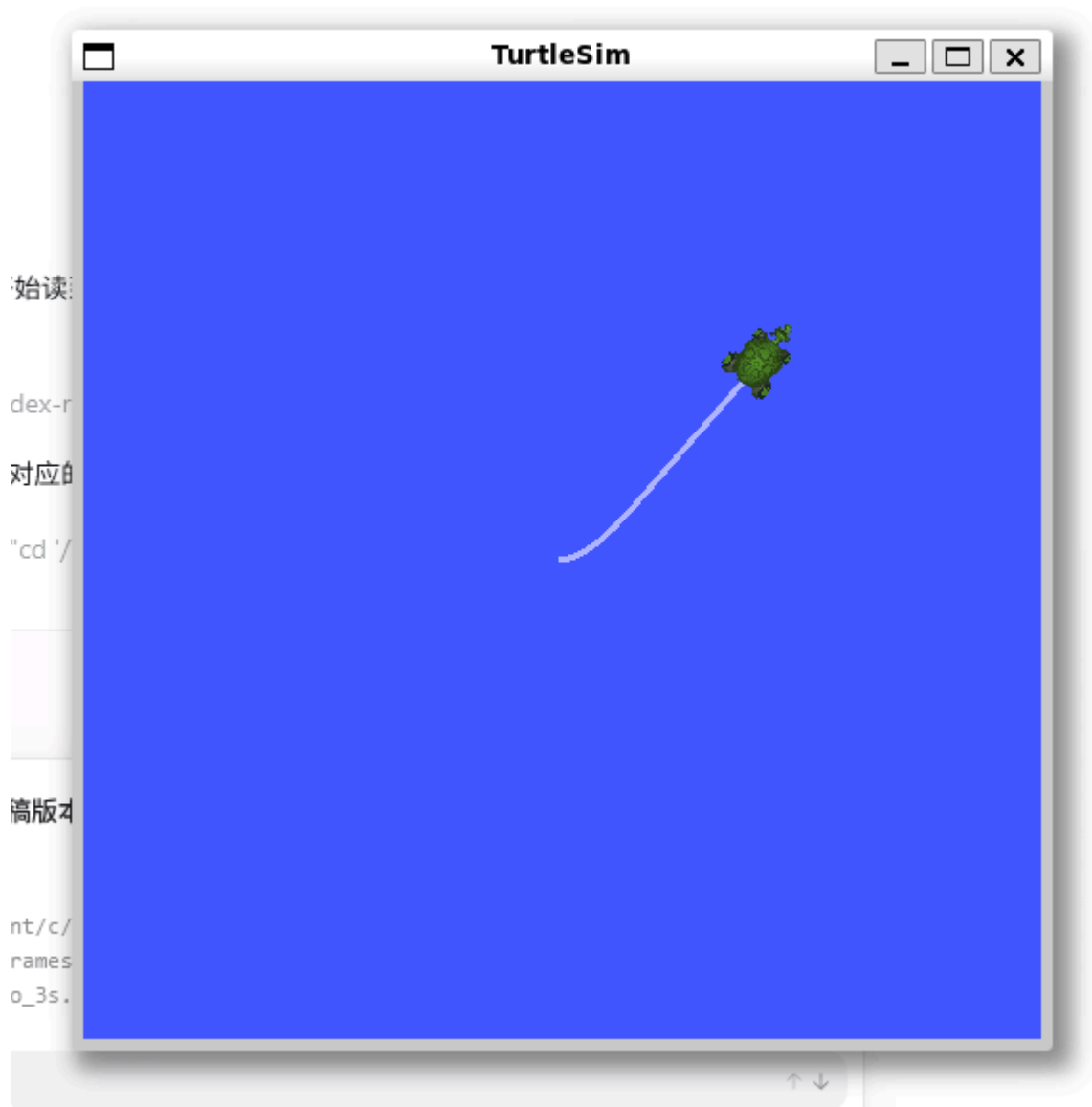
终端 2 — Action Server:

```
ros2 run py_action_tutorial move_to_position_server
```

终端 3 — Action Client:

```
ros2 run py_action_tutorial move_to_position_client
```

观察：海龟从初始位置 (5.54, 5.54) 向 (8.0, 8.0) 移动，终端 3 持续打印剩余距离，到达后输出最终误差。



上图为本次课件配套源码手动运行 `turtlesim_node`、`move_to_position_server` 与 `move_to_position_client` 后截取的真实实验窗口画面。

Step 5: 测试取消功能

在 Client 的 `__init__` 中初始化状态，并在 Goal 接受后保存 Handle:

```
self.goal_handle_ = None
self.cancel_sent_ = False

# goal_response_callback 中确认 accepted 后
self.goal_handle_ = goal_handle
```

然后在 `feedback_callback` 中，当剩余距离小于 2.0 时只发送一次取消:

```
def feedback_callback(self, feedback_msg):
    fb = feedback_msg.feedback
    self.get_logger().info(f"remaining: {fb.distance_remaining:.2f}")
    if (fb.distance_remaining < 2.0 and
        self.goal_handle_ is not None and
        not self.cancel_sent_):
        self.get_logger().info("Canceling goal...")
        self.goal_handle_.cancel_goal_async()
        self.cancel_sent_ = True
```

Result 回调必须检查 `wrapped_result.status == GoalStatus.STATUS_CANCELED`，不能把“收到 Result”一律打印成成功。

实验检查清单

检查项	验证方法
Action 已注册	<code>ros2 action list -t</code> 显示 <code>/move_to_position</code>
Server 收到 Goal	终端 2 打印目标坐标
Feedback 持续更新	终端 3 打印剩余距离递减
海龟正确移动	Turtlesim 窗口中海龟向目标行进
到达后停止	海龟停止运动，Result 显示误差 < 0.1
取消生效	取消后海龟立即停止，状态为 CANCELED

关键点

- 实战 Action 通常需要订阅传感器 Topic（如位姿）和发布控制 Topic（如速度）
 - Server 的 `execute_callback` 是控制循环的核心，Feedback 在每个控制周期发布
 - 取消时务必发送零速度指令，避免海龟继续运动
 - 本实验按单 Goal 操作；工程版应拒绝第二个活动 Goal，避免多个控制循环同时发布速度
 - 本实验是简化版导航，真实导航使用 `nav2` 的 `NavigateToPose` Action
 - 实验涵盖了 Action 的全部要素：Goal 校验、执行循环、Feedback、Result、取消
-

讲次 8: Action 命令行查看与调试

知识点

与 `ros2 topic` 和 `ros2 service` 类似，ROS 2 提供了 `ros2 action` 命令行工具集，用于发现、检查和手动测试 Action——无需编写代码即可验证 Action Server 是否正常工作。

ros2 action list — 列出所有 Action

```
ros2 action list
```

输出示例：

```
/count_until  
/move_to_position  
/turtle1/rotate_absolute
```

显示当前系统中所有活跃的 Action 名称。

查看 Action 名称和类型：

```
ros2 action list -t
```

输出：

```
/count_until [count_until_interfaces/action/CountUntil]  
/move_to_position [count_until_interfaces/action/MoveToPosition]  
/turtle1/rotate_absolute [turtlesim/action/RotateAbsolute]
```

`-t` 同时显示 Action 类型，确认 Server 使用了正确的接口。

ros2 action info — 查看 Action 详情

```
ros2 action info /count_until
```

输出：

```
Action: /count_until  
Action clients: 0  
Action servers: 1
```


字段	含义
Action clients	当前连接的 Client 数量
Action servers	当前提供服务的 Server 数量

正常状态: $\text{servers} \geq 1$ 。如果 $\text{servers} = 0$, 说明 Server 节点未启动或 Action 名称不匹配。

查看详细信息 (含节点名) :

```
ros2 action info /count_until -v
```

ros2 interface show — 查看 Action 类型定义

```
ros2 interface show count_until_interfaces/action/CountUntil
```

```
int64 target_number
float32 period
---
int64 reached_number
---
int64 current_number
```

手动构造 Goal 参数时, 必须清楚每个字段的类型和含义。

ros2 action send_goal — 手动发送 Goal

这是调试 Action 最强大的命令——直接在命令行发送 Goal, 观察 Feedback 和 Result:

基本语法:

```
ros2 action send_goal <action_name> <action_type> <goal_values> [options]
```

发送 CountUntil Goal:

确保 Server 正在运行, 然后:

```
ros2 action send_goal /count_until count_until_interfaces/action/CountUntil \
  "{target_number: 5, period: 1.0}"
```

输出 (实时刷新) :

```
Waiting for an action server to become available...
Sending goal:
  target_number: 5
  period: 1.0

Goal accepted with ID: ...

Feedback:
  current_number: 1

Feedback:
  current_number: 2
...
Feedback:
  current_number: 5

Result:
  reached_number: 5

Goal finished with status: SUCCEEDED
```

发送时查看 Feedback:

```
ros2 action send_goal /count_until count_until_interfaces/action/CountUntil \
  "{target_number: 10, period: 0.5}" --feedback
```

`--feedback` 显式打印每条 Feedback（部分版本默认已开启）。

测试 Turtlesim 旋转 Action:

```
# 终端 1
ros2 run turtlesim turtlesim_node

# 终端 2
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute \
  "{theta: 1.57}" --feedback
```

海龟将旋转到 1.57 弧度（90 度），终端显示 `remaining` 反馈。

测试 Goal 被拒绝:

```
ros2 action send_goal /count_until count_until_interfaces/action/CountUntil \
  "{target_number: -1, period: 1.0}"
```

输出:

```
Goal rejected
```

调试 workflow

当 Action 通信出问题时，按以下顺序排查：

1. `ros2 action list -t`
→ Action 是否存在? 类型是否正确?
2. `ros2 action info /my_action`
→ Server 数量是否 ≥ 1 ?
3. `ros2 interface show <action_type>`
→ Goal 字段名和类型是否正确?
4. `ros2 action send_goal ...`
→ 手动发送 Goal, Server 是否正常响应?
5. 检查代码中的 Action 名称是否一致
→ 代码中 "count_until" vs 命令行 `/count_until`
6. 检查节点是否正常运行
→ `ros2 node list` / `ros2 node info`

常见问题排查

现象	可能原因	解决方法
<code>ros2 action list</code> 为空	Server 未启动	运行 Server 节点
<code>send_goal</code> 一直等待	Server 未启动或名称不匹配	检查 Action 名称
Goal 被 REJECT	参数校验失败	检查 <code>goal_callback</code> 逻辑
无 Feedback 输出	<code>execute_callback</code> 未调用 <code>publish_feedback</code>	检查 Server 代码
Client 收不到 Result	未调用 <code>succeed()</code> / <code>abort()</code> / <code>canceled()</code>	检查 Server 结束逻辑
Feedback 有但海龟不动	Server 未发布 <code>cmd_vel</code>	检查 Publisher 逻辑

结合 `ros2 node info` 查看

```
ros2 node info /move_to_position_server
```

输出中包含该节点提供的 Action Servers 和订阅/发布的 Topic:

```
/move_to_position_server
Subscribers:
  /turtle1/pose: turtlesim/msg/Pose
Publishers:
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Action Servers:
  /move_to_position: count_until_interfaces/action/MoveToPosition
```

关键点

- `ros2 action list -t` 查看所有 Action 及其类型

- `ros2 action info <action>` 确认 Server / Client 数量
 - `ros2 action send_goal` 是手动测试 Action 的首选工具, 支持 `--feedback`
 - 排查问题时先命令行验证, 再调试代码
 - `ros2 interface show` 查看 Action 类型定义, 确保 Goal 字段正确
-

讲次 9：【C++】Action 编码挑战（独立完成）

知识点

本讲是独立的编码挑战。请根据以下需求描述，**从零独立实现** C++ Action Server 和 Client。讲师不提供完整代码，只提供需求、提示和验收标准。

挑战需求：TimerDelay Action

设计一个“延时完成”的 Action——Client 发送 Goal 请求 Server 等待指定秒数后返回结果。模拟机器人领域的“等待充电完成”、“等待电梯到达”等场景。

Action 定义： 在 `count_until_interfaces` 包中创建 `TimerDelay.action`：

```
# Goal
float32 wait_seconds      # 需要等待的总秒数
---
# Result
float32 elapsed_seconds   # 实际经过的秒数
bool completed            # 是否完整等待完成（取消则为 false）
---
# Feedback
float32 elapsed_seconds   # 当前已等待的秒数
float32 remaining_seconds # 剩余等待秒数
```

功能要求

Action Server (`timer_delay_server`)：

1. 接收 Goal 后，在 `goal_callback` 中拒绝 `wait_seconds <= 0` 或 `wait_seconds > 60` 的请求
2. 在 `execute` 中每秒发布一次 Feedback（当前已等待秒数 + 剩余秒数）
3. 支持取消：收到取消后停止等待，返回实际已等待时间，`completed = false`
4. 正常完成时返回 `completed = true`，`elapsed_seconds = wait_seconds`
5. 如果等待过程中节点被关闭，调用 `abort()`

Action Client (`timer_delay_client`)：

1. 启动后向 Server 发送 Goal：`wait_seconds = 15.0`
2. 在 Feedback 回调中打印进度百分比
3. 当 `elapsed_seconds >= 10.0` 时，自动发送取消请求
4. 在 Result 回调中打印最终状态和 `elapsed_seconds`
5. 完成后自动 shutdown

实现提示

- ▶ 提示 1：包结构（点击展开）
- ▶ 提示 2：Server 核心逻辑（点击展开）

- ▶ 提示 3: Client 取消逻辑 (点击展开)
- ▶ 提示 4: CMakeLists.txt 追加 (点击展开)

验收标准

#	测试项	预期结果
1	构建无错误	<code>colcon build --packages-select count_until_interfaces cpp_action_tutorial</code> 成功
2	Action 已注册	<code>ros2 action list -t</code> 显示 <code>/timer_delay</code>
3	正常完成	<code>send_goal "{wait_seconds: 5.0}"</code> → SUCCEEDED, completed=true
4	拒绝非法 Goal	<code>send_goal "{wait_seconds: -1.0}"</code> → rejected
5	拒绝过长 Goal	<code>send_goal "{wait_seconds: 100.0}"</code> → rejected
6	Feedback 更新	每秒打印 elapsed 和 remaining
7	取消功能	Client 在 10 秒时取消 15 秒 Goal → CANCELED, completed=false
8	最终状态	Result 回调正确打印 elapsed_seconds

运行验证

```
# 终端 1
ros2 run cpp_action_tutorial timer_delay_server

# 终端 2 - 命令行快速验证
ros2 action send_goal /timer_delay count_until_interfaces/action/TimerDelay \
  "{wait_seconds: 5.0}" --feedback

# 终端 2 - 完整 Client 测试 (含自动取消)
ros2 run cpp_action_tutorial timer_delay_client
```

进阶挑战 (可选)

完成基础要求后, 尝试以下扩展:

1. **多 Goal 排队**: Server 同时只执行一个 Goal, 其余在 `goal_callback` 中 REJECT 并打印 "Server busy"
2. **参数化 Client**: 通过 ROS 参数或命令行参数指定 `wait_seconds` 和取消阈值
3. **Python 版本**: 用 Python 实现相同的 Server 和 Client, 对比两种语言的 API 差异

关键点

- 独立实现是检验 Action 掌握程度的最好方式
- 挑战涵盖全部核心要素: Goal 校验、Feedback 循环、取消、Succeed/Abort
- 先实现最小可用版本, 再逐步添加取消和参数校验

- 用 `ros2 action send_goal` 快速验证 Server, 再调试 Client
 - 遇到问题时回顾讲次 3-6 的完整代码作为参考
-

本节总结

命令速查表

命令	说明
<code>ros2 action list</code>	列出所有活跃 Action
<code>ros2 action list -t</code>	列出 Action 及其类型
<code>ros2 action info <action></code>	查看 Action 的 Server / Client 数量
<code>ros2 action info <action> -v</code>	查看详细信息 (含节点名)
<code>ros2 action send_goal <action> <type> <values></code>	手动发送 Goal
<code>ros2 action send_goal ... --feedback</code>	发送 Goal 并显示 Feedback
<code>ros2 interface show <action_type></code>	查看 Action 类型定义
<code>ros2 interface list grep action</code>	列出所有已注册的 Action 类型
<code>ros2 node info <node></code>	查看节点提供的 Action Servers
<code>ros2 run py_action_tutorial count_until_server</code>	运行 Python Action Server
<code>ros2 run py_action_tutorial count_until_client</code>	运行 Python Action Client
<code>ros2 run cpp_action_tutorial count_until_server</code>	运行 C++ Action Server
<code>ros2 run cpp_action_tutorial count_until_client</code>	运行 C++ Action Client
<code>ros2 run py_action_tutorial move_to_position_server</code>	运行海龟移动 Server
<code>ros2 run py_action_tutorial move_to_position_client</code>	运行海龟移动 Client
<code>ros2 run cpp_action_tutorial timer_delay_server</code>	运行编码挑战 Server
<code>ros2 run cpp_action_tutorial timer_delay_client</code>	运行编码挑战 Client

核心 API 速查

操作	Python (rclpy.action)	C++ (rclcpp_action)
创建 Server	<code>ActionServer(node, Type, name, execute_cb, goal_callback=..., cancel_callback=...)</code>	<code>rclcpp_action::create_server<Type>(node, name, handle_goal, handle_cancel, handle_accepted)</code>
创建 Client	<code>ActionClient(node, Type, name)</code>	<code>rclcpp_action::create_client<Type>(node, name)</code>
发送 Goal	<code>send_goal_async(goal, feedback_callback=...)</code>	<code>async_send_goal(goal, send_goal_options)</code>
等待 Server	<code>wait_for_server()</code>	<code>wait_for_action_server(timeout)</code>
发布 Feedback	<code>goal_handle.publish_feedback(msg)</code>	<code>goal_handle->publish_feedback(msg)</code>
检查取消	<code>goal_handle.is_cancel_requested</code>	<code>goal_handle->is_canceling()</code>
标记成功	<code>goal_handle.succeed()</code>	<code>goal_handle->succeed(result)</code>
标记失败	<code>goal_handle.abort()</code>	<code>goal_handle->abort(result)</code>
标记取消	<code>goal_handle.canceled()</code>	<code>goal_handle->canceled(result)</code>
获取 Result	<code>goal_handle.get_result_async()</code>	<code>result_callback(WrappedResult)</code>
取消 Goal	<code>goal_handle.cancel_goal_async()</code>	<code>async_cancel_goal(goal_handle)</code>
并发执行策略	<code>MultiThreadedExecutor()</code> + 合适的 Callback Group	本例启动工作线程；也可使用受管理任务与 Executor 设计

三种通信机制对比表

维度	Topic	Service	Action
通信模式	发布/订阅 (Pub/Sub)	请求/响应 (Request/Response)	目标/反馈/结果 (Goal/Feedback/Result)
方向	单向	双向	双向
同步性	异步	同步 (Client 阻塞等待)	异步 (回调驱动)
持续时间	持续流 (无起止)	瞬间完成 (毫秒级)	长时间任务 (秒~分钟级)
返回值	无	有 (Response)	有 (Result)
进度反馈	无 (消息本身即数据)	无	有 (Feedback 持续发布)
可取消	不适用	无标准机制	内置 Cancel 支持
底层实现	RMW 发布/订阅 (默认常为 DDS)	RMW 请求/响应 (默认常为 DDS)	3 个 Service + 2 个 Topic 的高层组合
接口文件	<code>.msg</code>	<code>.srv</code>	<code>.action</code>
API 库	rclpy / rclcpp	rclpy / rclcpp	<code>rclpy.action</code> / <code>rclcpp_action</code>
QoS	可配置	默认 Service profile 通常为 Reliable/Volatile, API 可配置	由底层实体及 Action 选项决定, 不能视为单一固定策略
典型场景	传感器数据、速度指令、状态广播	查询参数、触发拍照、重置仿真	导航、抓取、机械臂运动、定时任务
ROS 2 CLI	<code>ros2 topic</code>	<code>ros2 service</code>	<code>ros2 action</code>
计算图显示	Publisher / Subscriber	Service Server / Client	Action Server / Client

选择口诀:

- 数据一直发 → Topic
- 一问一答瞬间完成 → Service
- 任务要一段时间, 还要知道进度 → Action

下一步

下一节 **2.5 自定义接口** 中, 我们将系统学习 ROS 2 接口类型的完整知识体系——`.msg`、`.srv`、`.action` 的高级用法、字段类型大全、嵌套与数组、接口包的版本管理与最佳实践。届时会把 2.2-2.4 中分散涉及的自定义接口内容统一串联, 并学习如何用 `ros2 interface` 工具链高效管理和调试所有自定义类型。